

M.L Lab programs

P1

```
from sklearn.datasets import fetch_california_housing
california = fetch_california_housing()
import pandas as pd
df = pd.DataFrame(california.data, columns=california.feature_names)
df["Target"] = california.target
import matplotlib.pyplot as plt
import seaborn as sns
df.hist(bins=30, edgecolor="black", figsize=(20,10))
plt.figure(figsize=(15,9))
for i, feature in enumerate(df.columns, 1):
    plt.subplot(3,3,i)
    sns.boxplot(df[feature])
    plt.tight_layout()
plt.show()
```

P2

```
from sklearn.datasets import fetch_california_housing

california = fetch_california_housing()

import pandas as pd

df = pd.DataFrame(california.data, columns=california.feature_names)
df["Target"] = california.target

correlation = df.corr()

import seaborn as sns
```

```
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
```

```
sns.pairplot(df)
```

P3

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
data = iris.data
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
scaler = StandardScaler().fit_transform(data)
```

```
pc = PCA(n_components=2).fit_transform(scaler)
```

```
import pandas as pd
```

```
df = pd.DataFrame(pc, columns=["pc1", "pc2"])
```

```
df["Target"] = iris.target
```

```
import seaborn as sns
```

```
sns.scatterplot(x="pc1", y="pc2", hue="Target", data=df)
```

P4

```
import pandas as pd
```

```
data = pd.DataFrame({  
    "Sky":["Sunny","Sunny","Rainy","Sunny"],  
    "AirTemp":["Warm","Warm","Cold","Warm"],  
    "Humidity":["Normal","High","High","High"],  
    "Wind":["Strong","Strong","Strong","Strong"],  
    "Water":["Warm","Warm","Warm","Cool"],  
    "Forecast":["Same","Same","Change","Same"],  
    "EnjoySport":["Yes","Yes","No","Yes"]  
})
```

```
print(data)
```

```
positiveData = data[data["EnjoySport"] == "Yes"].drop(columns="EnjoySport")
```

```
def findS(data):
```

```
    hypo = [0] * len(data.columns)
```

```
    for i in range(len(data)):
```

```
        for j in range(len(data.columns)):
```

```
            value = data.iloc[i, j]
```

```
            if hypo[j] == 0:
```

```
                hypo[j] = value
```

```
            elif hypo[j] != value:
```

```
                hypo[j] = "?"
```

```
    return hypo
```

```
finds = findS(positiveData)
```

```
print("Final Hypothesis:", finds)
```

P5

```
import numpy as np
```

```
x = np.random.rand(100).reshape(-1,1)
```

```
y = np.array([1 if val <= 0.5 else 2 for val in x[:50]])
```

```
xtrain = x[:50]
```

```
ytrain = y
```

```
xtest = x[50:]
```

```
kValues = [1,2,3,20,40]
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
plt.figure(figsize=(20,15))
```

```
for i, value in enumerate(kValues, 1):
```

```
    predicted = KNeighborsClassifier(n_neighbors=value).fit(xtrain,ytrain).predict(xtest)
```

```
    plt.subplot(5,2,i)
```

```
    plt.scatter(xtrain, ytrain, c="red", label="Tested")
```

```
    plt.scatter(xtest, predicted, c="blue", label="Predicted")
```

```
    plt.legend()
```

```
    plt.yticks(predicted)
```

```
plt.tight_layout()
```

```
plt.show()
```

P7

```
from sklearn.datasets import fetch_openml
```

```
boston = fetch_openml(name="boston", version=1)
```

```
import pandas as pd
```

```
df = pd.DataFrame(boston.data, columns=boston.feature_names)
```

```
df["Target"] = boston.target
```

```
x = df[["RM"]]
```

```
y = df["Target"]
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain,xtest, ytrain, ytest = train_test_split(x,y,test_size=0.2,random_state=42)
```

```
from sklearn.linear_model import LinearRegression
```

```
predict = LinearRegression().fit(xtrain,ytrain).predict(xtest)
```

```
import matplotlib.pyplot as plt
```

```
plt.scatter(xtest,ytest)
```

```
plt.plot(xtest,predict, c="red")
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
from sklearn.pipeline import make_pipeline
```

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv"
```

```
data = pd.read_csv(url)
```

```
data = data.dropna()
```

```
predict = make_pipeline(PolynomialFeatures(degree=2),  
LinearRegression()).fit(data[["horsepower"]], data["mpg"]).predict(data[["horsepower"]])
```

```
import seaborn as sns
```

```
plt.scatter(data["horsepower"], data["mpg"])  
sns.lineplot(x=data["horsepower"], y=predict, c="red")
```

P8

```
from sklearn.datasets import load_breast_cancer
```

```
cancer = load_breast_cancer()
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain,xtest, ytrain, ytest = train_test_split(cancer.data, cancer.target, test_size=0.2, random_state=42)
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
tree = DecisionTreeClassifier(criterion="gini").fit(xtrain,ytrain)
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(20,25))
```

```
plot_tree(tree,filled=True)
```

```
plt.show()
```

P9

```
from sklearn.datasets import fetch_olivetti_faces
```

```
faces = fetch_olivetti_faces()
```

```
faces.keys()
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain,xtest,ytrain,ytest, imgtrain,imgtest = train_test_split(faces.data, faces.target, faces.images,  
test_size=0.2, random_state=42)
```

```
from sklearn.naive_bayes import GaussianNB
```

```
prediction = GaussianNB().fit(xtrain,ytrain).predict(xtest)
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(20,15))
```

```
for i in range(10):
```

```
    plt.subplot(5,5,i+1)
```

```
    plt.imshow(imgtest[i])
```

```
    plt.title(f'Predicted: {prediction[i]} \nOriginal: {ytest[i]}')
```

```
plt.tight_layout()
```

```
plt.show()
```

10

```
from sklearn.datasets import load_breast_cancer
```

```
cancer = load_breast_cancer()
```

```
from sklearn.cluster import KMeans
```

```
predict = KMeans(n_clusters=2).fit_predict(cancer.data)
```

```
import matplotlib.pyplot as plt
```

```
plt.scatter(cancer.data[:,0],cancer.data[:,1],c=predict)
```

Prasanna